



Au cœur d'un service Windows qui fait du NoSQL

Partie 2

Maintenant que nous savons comment créer un service Windows, et que nous avons un thread pour vivre notre vie d'application principale, comment allons-nous faire pour mettre du code dans ce thread ? Et surtout on va y faire quoi ? Je vous propose de hoster un service REST Web API et d'exposer 2 méthodes get-data et set-data pour gérer une base NoSQL. C'est simple et ça peut servir...

Rappel de la fin de la partie I

Le code principal du service est ici :

```

UINT AutomateThread(LPVOID pParam)
{
    CString strLog;
    try
    {
        CoInitialize(NULL);

        // Add specific code here

        while (TRUE)
        {
            if (_Module.m_bStop)
            {
                break;
            }
            Sleep(200);

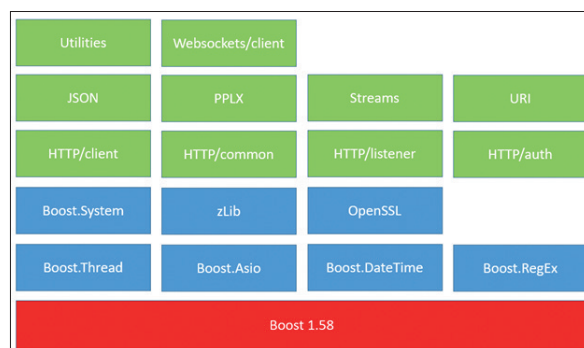
            // Main loop

        }
        catch (...)
        {
            strLog.Format(_T("EXCEPTION: Arrêt du service"));
            // _Module.LogEvent(strLog);
        }

        CoUninitialize();
        return 0;
    }
}

```

Nous sommes dans un thread qui sait gérer l'arrêt du service. Toute la plomberie est câblée. Maintenant, dans un service, on y met quoi ? On peut y mettre un point d'écoute comme un Pipe, une socket ou un handler http ? Je vous propose de mettre un service Rest Web API qui est autonome. Vous voulez dire une DLL que l'on charge sous IIS M'sieur ? Non, un serveur Rest complet qui gère le http dans sa totalité ! Ainsi on est autonome... Et comment on fait ça en C++ M'sieur ? On utilise le REST SDK de Microsoft qui permet de faire des applications portables REST JSON en C++.



1

Introduction au MS Rest SDK

Disponible sous GitHub à l'adresse <https://github.com/Microsoft/cpprestsdk>, ce SDK est basé du C++ ISO multiplateformes. Cela veut dire que vous pouvez builder sous Android, Windows, Linux et Mac.

Voici l'architecture de la solution (en rouge Boot, en bleu des modules et le code à compiler en vert): 1

Au travers ces noms de modules à compiler, on devine l'architecture de cette librairie. La librairie Boost permet d'être multiplateforme facilement et efficacement. Boost est reconnu dans la communauté C++ pour être à l'avant-garde de ce qui se fait de mieux. Voilà pour l'architecture. Pour ce qui est de l'utilisation, il faut maintenant compiler la librairie via Visual Studio pour x64 en Release et x64 en Debug. Une fois que c'est fait, on dispose d'un module debug de 12.5 MB et d'un module release de 6.3 MB. Donc le MS CPP REST SDK va nous permettre de hoster notre propre serveur Web avec des handlers http sur des ports et des verbes http. Génial !

On va hoster un service sur un port avec une URL donnée... Regardons ce qu'il se passe au lancement du serveur :

```

D:\Dev\GitHub_LMDB2\LMDBWindows\x64\Debug>myserver
08:39:42.324 - INFO - Init Master...
08:39:42.325 - INFO - IP : 192.168.175.241
08:39:42.408 - INFO - Worker node http://192.168.175.241:7001/MyServer/LMDB/
08:39:42.408 - INFO - Press ENTER to exit.

```

Dans notre cas, l'URL est /MyServer/LMDB et le port est 7001. On est capable de capter toutes les requêtes GET...

Reste à découvrir le code C++ qui permet de faire de si jolies choses... Si on regarde le code ci-dessus, on voit que l'adresse est

passée au constructeur de la classe. Ensuite on fait appel à la méthode Init(). Regardons ces deux bouts de code :

```
TheServer::TheServer(std::wstring url) : m_listener(url)
{
    std::function<void(http_request)> fnGet = &TheServer::handle_get;
    m_listener.support(methods::GET, fnGet);
    std::function<void(http_request)> fnPost = &TheServer::handle_post;
    m_listener.support(methods::POST, fnPost);
    std::function<void(http_request)> fnDel = &TheServer::handle_del;
    m_listener.support(methods::DEL, fnDel);
    std::function<void(http_request)> fnPut = &TheServer::handle_put;
    m_listener.support(methods::PUT, fnPut);

    this->_url = url;
}
```

On y aperçoit un membre m_listener à qui on passe une url et ensuite des pointeurs de fonctions sont passés à des méthodes http. Simple ! Le type m_listener est défini comme suit :

```
http_listener m_listener;
```

Voyons maintenant la méthode Init() :

```
void TheServer::Init()
{
    this->_http = std::unique_ptr<TheServer>(new TheServer(this->_url));
    this->_http->open().wait();
}
```

Quel est donc ce membre _http ?

```
std::unique_ptr<MyServer> _http;
```

C'est un smart pointeur sur la classe MyServer. Il y a une dernière méthode à expliquer :

```
pplx::task<void> open() { return m_listener.open(); }
```

Dans le thread, voici les étapes d'initialisation à ajouter – le code d'exécution est là :

```
DWORD AutomateThread(LPVOID pParam)
{
    CString strLog;
    try
    {
        ColInitialize(NULL);

        std::wstring port = Constants::MasterNodePort;
        std::wstring ip = ServerHelper::GetIP();
        std::wstring url = ServerHelper::BuildURL(ip, port);
        http::uri uri = http::uri(url);
        std::wstring address = uri.to_string();
        std::wstring name = _T("Master");
```

```
TCHAR sz[255];
    _stprintf_s(sz, _T("Node_%s_%s.log"), port.c_str(), name.c_str());
    g_Logger.Init(sz);
    g_Logger.WriteLog(_T("Init Node..."));

    //
    // Create the server node instance
    //

    _stprintf_s(sz, _T("IP : %s"), ip.c_str());
    g_Logger.WriteLog(sz);

    TheServer client(address);
    client._server = ip;
    client._port = port;
    client._name = name;
    client.Init();
    _stprintf_s(sz, _T("Worker node %s"), address.c_str());
    g_Logger.WriteLog(sz);

    while (TRUE)
    // ...{
```

Voilà, on peut voir que hoster un serveur REST est d'une facilité terrible...

Maintenant le corps du serveur REST, c'est sa méthode GET :

```
void TheServer::handle_get(http_request message)
{
    g_Logger.WriteLog(_T("handle_get"));

    PrintRequest(message);

    // http://192.168.175.241:7001/MyServer/LMDB/?request=set-data&key=toto0&value=toto1&name=cache2
    // http://192.168.175.241:7001/MyServer/LMDB/?request=get-data&key=toto2_k&name=cache3

    std::wstring request = ServerHelper::FindParameter(message, _T("request"));

    if (request == Constants::VerbPing)
    {
        RequestVerbPing(message);
        return;
    }

    if (request == Constants::VerbGetData)
    {
        RequestVerbGetData(message);
        return;
    }

    if (request == Constants::VerbSetData)
    {
        RequestVerbSetData(message);
```

```

    return;
}

message.reply(status_codes::OK);
}

```

Comme vous pouvez le voir, c'est propre. Une ou deux helper fonctions et hop le tour est joué !

```

std::wstring ServerHelper::FindParameterInQuery(std::map<std::wstring, std::wstring> query,
std::wstring var)
{
    auto it = query.find(var);
    std::wstring value;
    if (it != query.end())
    {
        value = it->second;
        std::wcout << _T("Extracted param ") << var << _T(": ") << value << std::endl;
    }
    return value;
}

std::wstring ServerHelper::FindParameter(http_request message, std::wstring var)
{
    auto query = uri::split_query(uri::decode(message.relative_uri().query()));
    std::wstring value = ServerHelper::FindParameterInQuery(query, var);
    return value;
}

```

Dans ma méthode GET, j'ai encapsulé les réponses que je fais à des verbes...

```

void TheServer::RequestVerbGetData(http_request message)
{
    USES_CONVERSION;
    CLMDBWrapper Imdb;

    g_Logger.WriteLog(Constants::VerbGetData.c_str());

    std::wstring key = ServerHelper::FindParameter(message, _T("key"));
    std::wstring dbNameW = ServerHelper::FindParameter(message, _T("name"));
    std::string dbName(dbNameW.begin(), dbNameW.end());

    if (Imdb.Init(LPSTR)dbName.c_str() == false)
    {
        g_Logger.WriteLog(_T("LMDB Init not done !"));
        message.reply(status_codes::OK);
        return;
    }

    char szKey[255];
    LPSTR lpszValue;

    strcpy_s(szKey, W2A(key.c_str()));

    if (Imdb.GetData(LPSTR)szKey, &lpszValue == true)

```

```

{
    Data data;
    data.key = key;
    data.value = std::wstring(A2W(lpszValue));

    free(lpszValue);

    TCHAR sz[255];
    _stprintf_s(sz, _T("Get Key:%s Value:%s"), data.key.c_str(), data.value.c_str());
    g_Logger.WriteLog(sz);

    std::wstring response = data.AsJSON().serialize();
    g_Logger.WriteLog(response.c_str());

    message.reply(status_codes::OK, data.AsJSON());
}
else
{
    message.reply(status_codes::OK);
}

Imdb.Uninit(LPSTR)dbName.c_str());
}

void TheServer::RequestVerbSetData(http_request message)
{
    USES_CONVERSION;
    CLMDBWrapper Imdb;
    g_Logger.WriteLog(Constants::VerbSetData.c_str());

    std::wstring key = ServerHelper::FindParameter(message, _T("key"));
    std::wstring value = ServerHelper::FindParameter(message, _T("value"));
    std::wstring dbNameW = ServerHelper::FindParameter(message, _T("name"));
    std::string dbName(dbNameW.begin(), dbNameW.end());

    if (Imdb.Init(LPSTR)dbName.c_str() == false)
    {
        g_Logger.WriteLog(_T("LMDB Init not done !"));
        message.reply(status_codes::OK);
        return;
    }

    LPSTR lpszKey = W2A(key.c_str());
    LPSTR lpszValue = W2A(value.c_str());
    Imdb.SetData(lpszKey, lpszValue);

    char sz[255];
    sprintf_s(sz, "Set Key:%s Value:%s", lpszKey, lpszValue);
    g_Logger.WriteLog(A2W(sz));

    Data data;
    data.key = A2W(lpszKey);
    data.value = A2W(lpszValue);

    std::wstring response = data.AsJSON().serialize();
    g_Logger.WriteLog(response.c_str());
}

```

```
message.reply(status_codes::OK, data.AsJSON());

lmdb.Uninit((LPSTR)dbName.c_str());
}
```

Vous le voyez, il ne faut pas beaucoup de code pour arriver au résultat voulu. Le code est plutôt simple à écrire. Vous remarquerez aussi que l'accès à la librairie LMDB est encapsulée dans la classe CLMDBWrapper. C'est important de construire des classes séparées. **2**

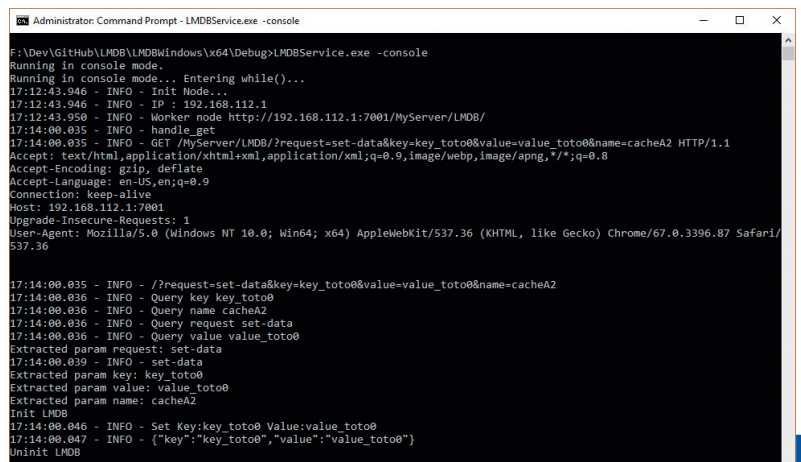
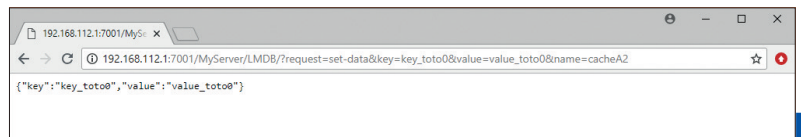
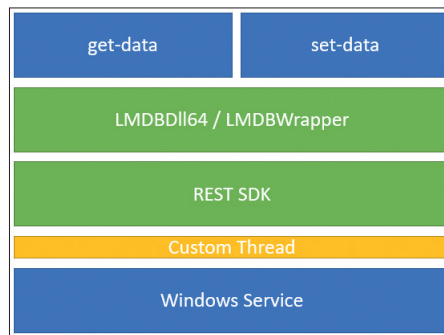
Pour tester le service Web, il suffit de lancer un browser... et de lancer cette url : http://192.168.112.1:7001/MyServer/LMDB/?request=set-data&key=key_toto0&value=value_toto0&name=cacheA2 **3**

Voici la log du service : **4**

Pour obtenir la valeur de la clé key_toto0, utilisons l'url suivante : http://192.168.112.1:7001/MyServer/LMDB/?request=get-data&key=key_toto0&name=cacheA2

Conclusion

Le service Windows est une application Windows pas toute faite comme les autres. Il faut s'exécuter dans un thread, savoir gérer un arrêt qui peut survenir n'importe quand. Un service Windows ne possède pas de code d'interface graphique, par contre c'est une boîte noire ; c'est de la communication. Cela nécessite des compétences « server » car on doit logger des choses et que on l'utilise en aveugle : ça fait le job tout seul, sans intervention humaine. •



reste cadre de 185 mm de large x 104 mm de haut